

A secure, live context graph

From arbitrary JSON and streamed media to temporal metrics, connected entities, and authorized experiences.

01 / IDENTITY & AUTHORIZATION

Workspace-wide reads • restricted entities need grants • writes require writer/admin

OIDC identity

Local development issuer · 2 replicas

RS256 JWT + HTTPS JWKS
APIs verify issuer, audience, subject, expiry
Development sign-in → 5-minute token

Production: managed OIDC + browser PKCE¹

Permission-check proxy

HTTPS · 2 replicas · separate API credential

Only POST /v1/permissions/check
Forces fully consistent SpiceDB checks
No policy writes with API credentials

Both Vert.x APIs connect through B

SpiceDB

v1.56.2 · 2 replicas

Workspace membership + entity relations
view / ingest / manage permissions
No positive permission cache; fail closed

Restrict, grant and revoke through operator only

PostgreSQL

Separate SpiceDB and Polaris databases

Verified TLS connection · single instance¹

Trusted operator tooling

permissions.py · schema + grants · admin credential

02 / INGEST, PROCESS & PERSIST

API, auth, Kafka, Polaris and S3 use verified TLS; identities and network policies limit access.

Producers

Clients + load generators

Unstructured JSON
PNG / JPEG images
Streamed video

Rate + concurrency controls
Schema drift / time disorder
GOP and streaming controls

Vert.x ingestion

v5.2 · 2 replicas · endpoint/topic config

JWT + workspace + ingest grant
Validate input and output JSON Schema
Derive canonical security labels

Media processing

Image dimensions, hashes, metadata
FFmpeg → ~2s keyframe HLS chunks

Acknowledge only after Kafka publish

M /media permission gate

Kafka

4.1.1 · 3 brokers · replication 3 / min ISR 2

ENDPOINT TOPICS

cg.secure.events
cg.secure.images
cg.secure.video

VALIDATED JSON + MEDIA REFERENCES

PROCESSED LIVE TOPICS

cg.secure.metrics / nodes / edges

Input video topic also feeds subscriptions ↓

Flink stream processing

2.3 · Kafka connector + Iceberg sinks

Revalidate schemas + canonical resource IDs
JSON Pointer projections and graph extraction
Event-time temporal aggregates + watermarks

Security scope survives every transform

Keys: workspace + resource + metric / window

2 JobManagers (active / standby) · 1 TaskManager
Parallelism 2 · isolated candidate jobs supported

Polaris + Iceberg

Polaris 1.7 · Iceberg 1.11 · RustFS object storage

Private catalog + warehouse

Polaris REST · 2 replicas · persistent catalog P
RustFS S3 + STS · context-warehouse bucket
events / nodes / edges / metrics; labeled rows
Reader and writer identities are separate
Vended, temporary table-scoped credentials

Flink commits · governed API reads

Private backends; no direct user credentials

03 / AUTHORIZED ACCESS

Error queue

Operational / admin only

cg.secure.errors

Schema / processing errors
Malformed / late records
Writers: ingestion + Flink
No GraphQL read access

Authorized failures only

Protected media store

RustFS context-media bucket + security sidecars

Images, playlists and .ts chunks
File → workspace + entity binding
Only ingestion holds media keys

Media bytes never enter Kafka,
Flink, Iceberg or DuckDB.

Vert.x GraphQL API

v5.2 · 2 replicas · only registered, labeled table schemas

LIVE / Kafka → Reactor Flux

Unique consumer group per API replica
read_committed · bounded per-client queues
Topic / entity routing + permission per event
Expiry, revocation, cancellation, reconnect

FEDERATED / DuckDB

iceberg + cache_https · Polaris schema discovery
Authorize EACH source BEFORE joins / aggregates
Credential-free joins; recheck contributors
Edges require access to BOTH endpoints

Context dashboard

Separate service · browser UI + Nginx proxy

Live metrics / graph / HLS video

GraphQL HTTP + graphql-transport-ws
Bearer token + workspace on queries / WS init
/auth/session → Secure, HttpOnly media cookie
/media → ingestion permission gate M

No direct file access · no data volume mounted

M MEDIA PATH

Browser / HLS.js ⇌ Nginx ⇌ ingestion authorization ⇌ RustFS media

Every file, playlist and range request rechecks access; active transfers check revocation and expiry.

04 / CONFIGURATION, RECOVERY & OPERATIONS

JSON / YAML configuration

Versioned, content-addressed ConfigMaps

ingestion.json + JSON Schemas → endpoint contracts
jobs.yaml → sources, transforms, windows, Kafka / Iceberg sinks
query.yaml + queries.yaml → schemas, SQL, GraphQL + routing

State & Flink orchestration

R Operator 1.16.1 (2 replicas) ↔ FlinkDeployment / jobs

Incremental RocksDB checkpoints · 10s interval / 5s min pause
Retained checkpoints + upgrade / hourly savepoints + HA metadata
RustFS recovery bucket · native savepoints · Kafka buffers input

Deployment & network controls

Kubernetes Secrets, service identities and enforced NetworkPolicies

Firewall-only kube-router · API → check proxy; direct SpiceDB / PG blocked
Rolling replicas + PDBs + readiness + request drain; schema/ACL provisioning
Candidate Flink jobs: isolate state, groups, transactions, topics and tables

CONNECTOR KEY Solid → data / media Dashed → control / authorization Matching A / B / C / E / M / P / R ports are explicit connections, repeated to keep routes readable.

¹ LOCAL PROFILE LIMITS

Single-node RustFS and single PostgreSQL are not HA. Savepoint upgrades pause job output. Production needs HA storage/DB, HTTPS ingress and OIDC PKCE.